Milestone 2 Report

Transformer Models, Embeddings, Similarity Search and Generative AI

Objective

The objective of this milestone was to understand the fundamental components of Transformer-based Natural Language Processing systems. The tasks focused on tokenization, contextual embeddings, attention mechanisms, semantic similarity, zero-shot classification, and generative language models.

---

1. Dataset Preparation

The dataset was loaded and processed using the Hugging Face datasets library. A new feature named `combined_text` was created by combining the question prompt and answer option text.

This helped understand how text preprocessing pipelines are created before passing data into Transformer models.

The transformation was performed using the `.map()` function, which is suitable for scalable dataset preprocessing.

---

2. BERT Tokenization Analysis

The `bert-base-uncased` tokenizer was initialized to study the WordPiece tokenization process.

Important tokenizer properties analyzed:

- Vocabulary size
- Special token IDs
- Input tensor representation

The vocabulary represents the complete collection of subword tokens learned during BERT pretraining.

Special tokens such as:

- `[CLS]`
- `[SEP]`
- `[PAD]`

help the Transformer understand sentence structure and boundaries.

3. Batch Tokenization

The complete prompt column was tokenized using:

- Fixed maximum sequence length
- Padding
- Truncation
- PyTorch tensor conversion

Configuration:

```
padding="max_length"
truncation=True
max_length=128
return_tensors="pt"
```

The resulting tensor demonstrated how textual data is converted into numerical input suitable for Transformer models.

---

4. BERT Architecture Understanding

The internal architecture of BERT was analyzed.

BERT-base configuration:

- Hidden dimension: 768
- Attention heads: 12

Each attention head receives a smaller projection of the embedding space.

The attention head dimension is calculated as:

```
768 / 12 = 64
```

This allows multiple attention heads to learn different semantic relationships.

---

5. Contextual Embeddings

A pretrained BERT model was loaded using:

```
AutoModel.from_pretrained()
```

The prompt from the dataset was passed through the model.

The output tensor:

```
last_hidden_state
```

contains contextual embeddings for every token.

The `[CLS]` embedding was extracted because it acts as the sentence-level representation in BERT.

---

6. Attention Mechanism Exploration

BERT was loaded with:

```
output_attentions=True
```

Attention matrices were extracted to analyze how tokens interact.

The experiment measured how much attention the `[CLS]` token assigned to another token.

This demonstrated that Transformer models dynamically learn relationships between words depending on context.

---

7. Sentence Embedding Similarity

The Sentence Transformer model:

```
sentence-transformers/all-MiniLM-L6-v2
```

was used to generate dense vector representations.

Cosine similarity was calculated between:

- Question prompt
- Answer options

using:

```
sentence_transformers.util.cos_sim()
```

This showed how semantic similarity can be measured beyond simple keyword matching.

---

8. Ranking Pipelines Comparison

Two ranking systems were implemented.

Pipeline 1: TF-IDF Retrieval

Traditional sparse representation:

- Term frequency based
- Keyword matching
- Cosine similarity ranking

Pipeline 2: MiniLM Embedding Retrieval

Modern dense retrieval:

- Semantic embeddings
- Transformer-based understanding
- Cosine similarity ranking

Performance was evaluated using:

```
MAP@3
```

MiniLM performed better because semantic embeddings capture meaning rather than only word overlap.

---

9. Zero-shot Classification

The model:

```
facebook/bart-large-mnli
```

was used for zero-shot classification.

Experiments compared:

- Softmax probability ranking

- Independent sigmoid scores using multi_label=True

This showed different probability interpretation methods.

---

10. Generative AI Experiment

A text-to-text generation model:

```
google/flan-t5-small
```

was used.

A structured prompt was provided and the model generated the answer choice directly.

This demonstrated how generative models solve tasks through instruction following instead of fixed classification heads.

Conclusion

This milestone provided practical understanding of Transformer NLP pipelines including:

- Tokenization
- Embedding generation
- Attention visualization
- Semantic similarity
- Zero-shot inference
- Generative AI

It demonstrated the evolution from traditional TF-IDF approaches to modern Transformer-based language understanding.